

MMoE 与 GradNorm：从第一性原理推导多任务学习

一份保留思维连续性的推导笔记

写作原则

这不是从“标准架构图”倒推的讲义，而是按“问题起点 → 自己搭结构 → 撞坑/纠偏 → 收敛”的顺序整理的推导过程。MMoE 的前向、GradNorm 的整套机制（相对训练速度、target、 L_{grad} 、只更新 w_k ）在正文里都是一步步推出来的，不是背下来的。中间几处“岔路”（共享 expert 是不是 shared-bottom、sparse vs soft、 w_k 能不能预定义、梯度投影 vs 梯度范数）被特意保留，因为二义性正是从这些地方来的。

目录

1. 问题起点：多任务学习要解决什么
 2. 架构演进：Shared-Bottom → MoE → MMoE
 3. 岔路一：共享 expert 是不是 shared-bottom?
 4. 岔路二：soft (MMoE) vs sparse (LLM MoE)，数学差多远?
 5. MMoE 前向推导： $H = GY$
 6. 输入层：shared embedding, x 从哪来
 7. 为什么需要 GradNorm：多任务的梯度失衡
 8. 岔路三：梯度冲突（投影）vs 梯度失衡（范数）——两条路
 9. GradNorm 推导（一）：相对训练速度 r_k
 10. 岔路四： w_k 能不能预定义？scale 能不能提前排？
 11. GradNorm 推导（二）：target 与 L_{grad}
 12. GradNorm 推导（三）：只更新 w_k ，target 要 detach
 13. 完整闭环与训练流程
 14. 这次推导训练到的东西
-

1. 问题起点：多任务学习要解决什么

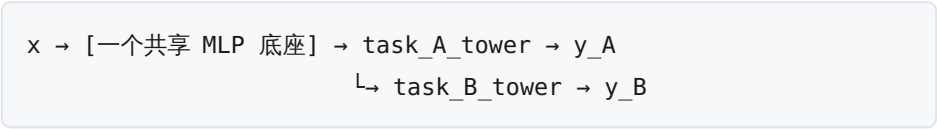
推荐/排序系统里，一个模型常要同时优化多个目标：CTR（点击）、CVR（转化）、时长、点赞……这些是多个 task，共用同一批输入特征。

核心张力：这些 task 之间既相关（都关于”用户对内容的兴趣”）又冲突（点击多的不一定转化高）。我们想要：- **共享：**让相关的 task 互相帮助（知识迁移）；- **不互相拖累：**冲突的 task 不要因为共享而互相损害（负迁移，negative transfer）。

多任务架构的演进，就是在”共享多少、怎么共享”上做文章。

2. 架构演进：Shared-Bottom → MoE → MMoE

第一代：Shared-Bottom



底座是**单个**共享 MLP，产出一个表示 h ，所有 task 拿到**同一个** h ，再各自进 tower。**没有 gate**。问题：底座被迫服务所有 task，冲突时两头不讨好（负迁移严重）。

第二代：MoE (single-gate) 把单一底座换成”多个 expert + 一个共享 gate”。expert 分工了，但所有 task 共用同一个 gate，还是被迫用同样的 expert 组合。

第三代：MMoE (multi-gate) expert 共享，但**每个 task 一个 gate**。冲突的 task 可以通过各自的 gate，从共享 expert 池里挑不同的组合，各取所需。“multi-gate”的 multi 指的是多个 gate（每 task 一个），不是多个 expert。

3. 岔路一：共享 expert 是不是 shared-bottom?

推导中一个自然的疑问：“MMoE 里 N 个 expert 对所有 task 都是同一批，这不就是 shared-bottom 吗？只是底座从一个 MLP 换成 N 个 expert？”

这个直觉抓住了”共享精神一脉相承”（对），但把两者等同是错的。关键区别：

	Shared-Bottom	MMoE 共享 expert
底座	1 个 MLP	N 个 expert
产出	1 个表示 h	N 个表示 $e_1 \dots e_N$
每个 task 拿到	同一个 h （强制一致）	各自加权组合（可不同）
能否”挑”	不能	能，gate 决定配比

- Shared-Bottom 的”共享” = 所有 task **被迫**用同一个输出。
- MMoE 的”共享 expert” = expert 池**对所有 task 开放**，但每个 task 用**自己的 gate**组合出**不同的**表示。

验证（关键）：若所有 task 的 gate 都输出相同权重（比如均匀 $1/N$ ），MMoE 就退化成“所有 task 拿同样组合” $\approx$ shared-bottom。正是” 每 task 不同 gate” 让它区别于 shared-bottom。去掉 multi-gate, MMoE 塌回 shared-bottom。

比喻：shared-bottom 是食堂只有一个大锅菜，所有人吃一样的；MMoE 是有 N 个菜的共享菜品池，每个人（task）按自己口味（gate）打不同配比——菜共享，但餐盘不同。

结论：共享的是 expert 池，不同的是每 task 的 gate 配比。“共享 = 池子开放”，不是” 共享 = 强制一致”。MMoE 的创新点是 per-task gate，不是” 底座换成多 expert” ——把它说成” shared-bottom 换底座” 会丢掉创新点。

4. 岔路二：soft (MMoE) vs sparse (LLM MoE) ，数学差多远？

另一个自然的判断：“sparse MoE 更好吧，inference 只激活 top-k，省算力、降 latency。” —— 在 LLM 场景对（Switch/Mixtral 就是为了大容量+省算力）。但要看清 MMoE 和 sparse MoE 解决的是不同问题：

	Sparse MoE (LLM)	MMoE (多任务)
目的	扩容量、省算力	解决任务冲突
expert 数	多 (64+)	少 (4-8)
gate/router	每 token 一个 router	每 task 一个 gate
激活	top-k 稀疏	全部 soft 加权

“数学推导差多远？” —— 几乎不差，只差一个求和范围。两者前向完全相同（N 个独立 expert、gate softmax），唯一分叉在结合那步：

$$\text{MMoE(soft): } h = \sum_{i=1}^N g_i e_i(x)$$

$$\text{Sparse: } h = \sum_{i \in \text{TopK}(g)} \tilde{g}_i e_i(x)$$

但这一个 TopK 带来两个不平凡的后果（sparse 的真实成本）：1. TopK 不可导：没被选中的 expert 拿不到梯度 $\rightarrow$ expert collapse（少数 expert 垄断）。2. 需要 load balancing 辅助 loss：逼 gate 均匀使用各 expert ($L_{aux} = \alpha N \sum_i f_i P_i$)。MMoE 全激活，不需要这些。

为什么多任务默认用 MMoE soft: expert 本来就少（全激活开销小，省不了多少 latency）；多任务要 soft 配比（top-k 硬选丢软信息）；没有 collapse/load-balancing 的训练难题。

术语精确：一 task 独占一 expert（任务A固定用expert1）不是 MoE，是模型集成（ensemble）。MoE 的核心是” expert 池共享 + gate 配比”，不是独占。

5. MMoE 前向推导: $H = GY$

现在从零推前向。设 K 个 task、 N 个 expert。

Expert (共享池) : 每个 expert 是一个 MLP:

$$y_i = \sigma(W_i x + b_i), \quad i = 1..N$$

x 是 $(1, d_x)$, y_i 是 $(1, d_y)$, 故 W_i 是 (d_x, d_y) 。

Gate (每 task 一个) :

$$g_k(x) = \text{softmax}(W_g^k x + b_g^k), \quad k = 1..K$$

gate 输出维度是多少? —— 被后面的运算锁死为 N 。 这不是自由参数: gate 要给 N 个 expert 各一个权重, 所以 $g_k(x) \in \mathbb{R}^N$ (W_g^k 是 (d_x, N))。gate 内部隐藏层可自由设计, 但**出口必须 = N** (对 N 个 expert 配权重)。就像分类器最后一层 = 类别数。

这一点在推导里最好自己从矩阵乘法维度相容推出来, 而不是背: “下一步 $h_k = g_k(x) [y_1 \dots y_N]^T$ 要成立, g_k 的列数必须 = expert 数 N 。” 运算逼出维度, 不是预设。

加权组合 (MMoE 的核心操作) : task k 用自己的 gate 权重组合 N 个 expert:

$$h_k = \sum_{i=1}^N g_k(x)_i \cdot y_i = g_k(x) [y_1; \dots; y_N]$$

矩阵形式: 设 G 是 (K, N) (每行一个 task 的 gate 权重)、 Y 是 (N, d_y) (每行一个 expert 输出) :

$$\boxed{H = GY}, \quad H \text{ 是 } (K, d_y)$$

H 的第 k 行 h_k = task k 的融合表示。每个 task 的 h_k 不同, 因为 G 每行不同——这就是 multi-gate 的核心。

Tower (task-specific) :

$$\hat{y}_k = \text{tower}_k(h_k)$$

tower 结构随任务定: MLP (最常见)、RNN/LSTM (序列)、Transformer (attention)、CNN——任意 deep NN。通常不深 (表示学习的重活在 expert 层已做)。

6. 输入层: shared embedding, x 从哪来

一个工业界落地的补充: 上面的 x 不是原始输入, 而是共享 embedding 层的输出。原始特征是异构的:

- **sparse feature** (user_id、item_id、category) $\rightarrow$ embedding table $\rightarrow$ 稠密向量

- **dense feature**（价格、时长、统计量）→ normalize / bucketize
- **pretrained embedding** → 直接接入
- 全部 concat/pooling → 统一 embedding x , $(1, d_x)$

原始异构特征 → [shared embedding layer] → x → N expert + K gate
→ H=GY → K tower

术语：这层叫 **shared embedding layer / input layer**，别叫“shared-bottom”（后者特指被 MMoE 取代的单一 MLP 底座，见第 3 节）。这里的“shared”指所有 expert/gate 共用同一套 embedding，是**比 expert 更底层的全局共享**——sparse embedding table 是参数量大头（user/item 上亿），改它影响所有 task。

7. 为什么需要 GradNorm：多任务的梯度失衡

总 loss 最朴素的写法：

$$L_{total} = \sum_{k=1}^K w_k L_k$$

每个 L_k 按 task 定义（CE / MSE / ...）。

问题：不同 task 的**梯度 scale 差很多**（loss 量级不同，比如 CTR 的 CE ≈ 0.5 ，某回归 MSE ≈ 100 ）。若 w_k 固定，**大梯度的 task 会主导训练**，小 task 学不动——优化被拽向大 task，其余任务欠拟合。

目标：动态调 w_k ，让各 task 对共享参数的**梯度平衡**，不让任何一个主导。好处直观：避免优化路径被大梯度 task 拽得剧烈震荡，各 task 平稳地一起收敛。

关键定位：失衡发生在**共享部分**（expert/embedding/共享层）——多 task 的梯度在这里“打架”。task-specific 的 tower 各管各的，不打架。所以要平衡的是**各 task 对共享参数 W 的梯度**。

W 通常取**共享部分的最后一层**（代表性足够、又省算力，算全部共享参数的范数太贵）。

8. 岔路三：梯度冲突（投影）vs 梯度失衡（范数）——两条路

推导中一个很自然的 brainstorm：“让各 task 的梯度在某个方向上互不影响、往左往右变化为 0，保证一个梯度不影响另一个。”

这个直觉很敏锐，但它指向的是另一支技术，不是 GradNorm：

- 你 brainstorm 的（梯度投影 / PCGrad）：处理梯度方向冲突——两个 task 梯度 g_i, g_j 若 $g_i \cdot g_j < 0$ （冲突），把 g_i 投影到 g_j 的正交补：
 $g'_i = g_i - \frac{g_i \cdot g_j}{\|g_j\|^2} g_j$ ，消除冲突分量。这正是”在那个方向上变化为 0”的精确形式。关键词：方向、投影、内积、正交。
- GradNorm：处理梯度大小失衡——让各 task 梯度的范数平衡（按训练速度）。关键词：范数、大小、训练速度。

两条路解决不同问题：一个管方向冲突，一个管大小失衡。之所以推导时会拧着，是因为”映射到某个方向、在方向上变化为 0”是投影法的语言（方向），而 GradNorm 是范数的语言（大小）。

本笔记推 GradNorm（范数平衡）。（PCGrad 是另一份 note 的事。）

9. GradNorm 推导（一）：相对训练速度 r_k

要平衡梯度大小，先要一把跨 task 可比的”学习速度”尺子。

第一步：消量级。不同 task 的 loss 绝对值不可比（量级差异）。用相对起点的比值：

$$\tilde{L}_k(t) = \frac{L_k(t)}{L_k(0)}$$

当前 loss / 初始 loss。无量纲，消除了量级差异。 $\tilde{L}_k$ 小 = loss 降得多 = 学得快；大 = 学得慢。

这里要用比值，不是当前 loss L_k 本身。若直接用 $L_k / \sum L_j$ ，会被 loss 量级主导（大量级 task 永远占比大），和”学习速度”无关。必须先除以初始 loss 消量级。

第二步：归一化（跨 task 可比）。

$$r_k(t) = \frac{\tilde{L}_k(t)}{\frac{1}{K} \sum_j \tilde{L}_j(t)}$$

r_k = task k 的比值 ÷ 所有 task 平均比值 = 相对训练速度。

- $r_k > 1$ ：比平均慢； $r_k < 1$ ：比平均快。
- 现在 $r_A > r_B$ 明确表示”A 比 B 学得慢”，不受 loss 单位干扰——可比了。

分母那个 $\frac{1}{K}$ 是可选缩放：去掉它（用总和）， r_k 变成”占比”（和为 1）；带上它， r_k 变成”相对平均”（围绕 1）。本质是归一化， $\frac{1}{K}$ 只影响语义（占比 vs 相对平均），不影响相对关系——常数因子后续会被吸收。

10. 岔路四： w_k 能不能预定义？scale 能不能提前排？

推导中一个反复出现的想法：“给每个 task 的梯度范数一个 scale， $\|w_k \nabla_W L_k\| \leq \text{scale}$ ，这个 scale 可以提前定义——比如头 100 步 0.1、100–1000 步 0.05，10 个 task 就 scale/10 均分。”

这个方案能跑、自治，但它不是 GradNorm，有两个本质区别（也是 GradNorm 的两个创新）：

区别 1：均分 vs 按训练速度不均分。“scale/10 均分”等于假设“所有 task 该有相同梯度范数”。但 GradNorm 的核心恰恰是**不均分**——学得慢的 task 该给更大 target（推它），快的给小的。均分抹掉了 task 间学习速度差异，而处理这个差异正是 GradNorm 存在的理由。

区别 2：预设时间表（开环）vs 实时算基准（闭环）。“按步数排好的 scale”是开环：不看每个 task 的真实状态，按预定时间表走，还要人工调这个时间表。GradNorm 用“当前所有 task 梯度范数的平均”当基准，闭环自适应，不用预排。

关键的思路转换：别想着“提前给 scale”，而是问——“此刻，一个 task 的梯度范数该多大才算‘合群’？答案是：向此刻所有 task 的平均看齐。”这个平均是实时算的，不是提前定的。

另一个纠偏： w_k 不能提前定义死。 w_k 正是 GradNorm 要动态学习的量——若预定义，就退化成普通固定加权，不需要 GradNorm 了。 w_k 是被一个 loss 优化出来的（第 11–12 节），不是解析式。

11. GradNorm 推导（二）：target 与 L_{grad}

有了尺子 r_k ，现在构造 target 和优化目标。

每个 task 的梯度范数（要平衡的量）：

$$G_W^{(k)} = \|\nabla_W(w_k L_k)\|_2 = \|w_k \nabla_W L_k\|_2$$

task k 的加权 loss 对共享参数 W 的梯度范数。 w_k 乘进去了。

target 梯度范数：

$$\text{target}_k = \bar{G}_W \cdot r_k^\alpha, \quad \bar{G}_W = \frac{1}{K} \sum_k G_W^{(k)}$$

- $\bar{G}_W$ （当前平均范数）= 基准量级（实时算，不预设，提供“绝对大小”）。- r_k^α = 相对调整： $r_k > 1$ （慢）→ target 大（推它）； $r_k < 1$ （快）→ target 小。 α 控制推的力度。

为什么 target 要乘 $\bar{G}_W$ 、不能直接 $= r_k$? 因为 r_k 是无量纲的 (量级 ≈ 1)，而梯度范数有真实量级。直接令范数 $= r_k$ 会把范数错误地钉在 1 附近。乘 $\bar{G}_W$ 提供量级， r_k 只管相对调整。而 $\bar{G}_W = \frac{1}{K} \sum \|w_k \nabla_W L_k\|$ 本身就是“当前加权梯度范数的平均”——自带量级。

L_{grad} : 逼实际范数趋近 target。用平方距离 (MSE) 或 L1:

$$L_{grad} = \sum_k (G_W^{(k)} - \text{target}_k)^2$$

(原论文用 L1 $\sum_k |G_W^{(k)} - \text{target}_k|$; MSE 也可, 方向一致。)

它是一个独立的辅助目标, 不加入到原始 cost 上。这是 GradNorm 的关键结构特征: $-L_{total} = \sum_k w_k L_k \rightarrow$ 更新网络参数 θ ; $-L_{grad} \rightarrow$ 只更新权重 w_k , 不碰网络。

它不是“加进主 loss 的正则项” (那种和网络参数一起求导), 而是独立优化 w_k 的目标。

12. GradNorm 推导 (三): 只更新 w_k , target 要 detach

w_k 怎么更新? —— L_{grad} 对 w_k 求导, 梯度下降:

$$w_k \leftarrow w_k - \eta \frac{\partial L_{grad}}{\partial w_k}$$

- r_k 大 (慢) $\rightarrow$ target 大 $\rightarrow$ 实际 $G_W^{(k)} < \text{target} \rightarrow L_{grad}$ 推 w_k 变大 $\rightarrow$ 慢 task 权重增大。
- 这和“ w_k 正相关于 r_k ”一致: 慢的给大权重。(注意不是“直接倒数”——直接令 $w_k = L_0/L_k$ 方向会反; w_k 是被 L_{grad} 优化出来的, 不是解析式。)

两个关键工程细节 (最容易错):

1. 算 $G_W^{(k)}$ 时要保留计算图对 w_k 可导。即 $G_W^{(k)} = \|w_k \nabla_W L_k\|$ 里, w_k 在范数内, 用 `autograd.grad(..., create_graph=True)`, 这样 L_{grad} 才能对 w_k 求导。
2. target 要 detach (当常数)。target 里含 $\bar{G}_W$ (也含梯度范数), 但求 $\partial L_{grad}/\partial w_k$ 时, target 视为常数, 只让实际那项 $G_W^{(k)}$ 对 w_k 求导。否则梯度乱套、训练不稳。
3. renormalize: 每步后把 w 归一化使 $\sum_k w_k = K$ (防止所有 w_k 一起漂走, 等价于改了全局 learning rate)。

13. 完整闭环与训练流程

每一步:

1. 前向: $H = GY$, 各 task $\hat{y}_k = \text{tower}_k(h_k)$, 算 L_k 。

2. 主更新: $L_{total} = \sum_k w_k L_k$, `backward` 更新网络参数 θ (w_k 当常数)。
3. 算训练速度: $\tilde{L}_k = L_k(t)/L_k(0)$, $r_k = \tilde{L}_k/\bar{\tilde{L}}$ 。
4. 算梯度范数: $G_W^{(k)} = \|w_k \nabla_W L_k\|$ (对共享 W , `create_graph=True`)。
5. target: $\text{target}_k = \bar{G}_W \cdot r_k^\alpha$ (`detach`)。
6. GradNorm 更新: $L_{grad} = \sum_k (G_W^{(k)} - \text{target}_k)^2$, `backward` 只更新 w_k 。
7. renormalize: $w \leftarrow w \cdot K / \sum_k w_k$ 。

效果: 固定权重下大 task 主导、小 task 学不动; GradNorm 下 w 自动调整 (大 task 权重 $\downarrow$ 、小 task 权重 $\uparrow$) , 两 task 梯度平衡、一起收敛。

14. 这次推导训练到的东西

1. MMoE 的创新在 multi-gate, 不在多 expert: 共享 expert 池 + 每 task 不同 gate 配比 $\rightarrow$ 缓解负迁移。去掉 multi-gate 塌回 shared-bottom。
2. soft vs sparse 前向只差求和范围 (全部 vs top-k) , 但 TopK 不可导引出 collapse/load-balancing 的工程复杂度——这才是本质区别。
3. 维度由运算锁死, 不预设: gate 出口 = N, 是被 $H = GY$ 的矩阵相容性逼出来的。
4. GradNorm 平衡的是梯度范数 (大小) , 不是 loss 大小; 和梯度投影 (方向冲突) 是正交的两条路。
5. 训练速度要用比值 $L_k(t)/L_k(0)$ 消费量级, 再归一化才可比。
6. 不预定义 scale / 不预定义 w_k : GradNorm 的精髓是”实时算基准 (当前梯度范数平均) + 按训练速度不均分” , 而非”预排时间表 + 均分”。
7. 双 loss 各管各的参数: L_{total} 更新网络、 L_{grad} 只更新 w_k ; target 要 detach。

一句话压缩: MMoE = 共享 expert 池 + 每 task 一个 gate (soft 加权 $H = GY$) + task tower; GradNorm = 用”当前梯度范数平均 $\times$ 相对训练速度 r_k “定 target, 构造独立的 L_{grad} 只优化任务权重 w_k (target detach) , 让各 task 对共享参数的梯度范数动态平衡、不均分、不预设。